

MODULE 03

上下文、提示词、记忆、压缩与缓存

Claude Code 的核心资产负债表：什么进入窗口、何时加载、如何缩减

高质量 agent 的首要问题不是“prompt 写得多漂亮”，而是每轮模型实际看到了什么、这些内容何时加载、是否稳定命中缓存，以及丢失时如何恢复。

研究基线：Claude Code v2.1.236（稳定版） | Python Agent SDK v0.2.141（tag） | 历史源码观察 v2.1.88

资料核验日期：2026-08-19 | 中文深度研究版

目录

- 1. 九类有序上下文来源
- 2. Prompt 不是一个字符串，而是片段编译器
- 3. CLAUDE.md、Rules 与 Auto Memory
- 4. Skills 是渐进式上下文加载器
- 5. 五阶段压缩管线
- 6. 摘要的目标不是短，而是可继续
- 7. Prompt caching：稳定前缀是第一性能指标
- 8. Context budget 实战
- 9. 常见反模式

阅读提示：内部模块名以历史源码观察解释，当前功能以官方文档与 v0.2.141 SDK 源码校验。

1. 九类有序上下文来源

顺序	来源	加载时机	主要成本/风险
1	基础 system prompt	会话开始	最大缓存前缀，版本变化会失效
2	output style / model/provider sections	开始	条件分支造成 cache key 变化
3	managed/user/root CLAUDE.md	开始	长期占用；过长降低遵循
4	project rules	开始或路径匹配	规则命中时机影响轨迹
5	skill descriptions	发现阶段	目录大时描述也占 token
6	tool schemas	每次请求的可用工具池	MCP 数量最易膨胀
7	conversation/messages	持续增长	文件与工具输出是最大变量
8	nested CLAUDE/rules/hooks	读文件/事件时动态注入	隐式、容易重复
9	auto memory/topic files	索引启动、详情按需	陈旧知识和跨会话污染

顺序决定缓存：越稳定、越通用的内容越靠前；会话历史和工具输出放在后部。动态段落如果插入前缀中间，会导致后续整段 cache miss。 [O8, O9, R2]

2. Prompt 不是一个字符串，而是片段编译器

历史源码与提取目录显示 system instructions 按能力、平台、模式、工具、模型和实验开关拆成大量片段。组装器选择条件片段、插入环境与工具描述，再输出 Messages API 的 system blocks。这个设计让功能可组合，却也容易形成互相重复或相互冲突的提醒。 [R2, R3]

- 身份与行为原则：软件工程焦点、沟通风格、安全边界。
- 运行时事实：cwd、平台、git 状态、模型/provider、日期。
- 工具政策：何时用 Read/Grep/Bash/Agent/Skill，参数语义与限制。
- 模式段：plan、auto、coordinator、subagent、remote、learning/output style。
- 事件提醒：文件截断、hook 结果、plan approved、task notification、compaction continuation。

源码分析方法

不要复制“完整 system prompt”当作架构。真正要追的是：片段的触发条件、插入顺序、缓存位置、作用域、重复消除，以及它与确定性权限检查的边界。

3. CLAUDE.md、Rules 与 Auto Memory

机制	定位	启动加载	修改生效
Managed CLAUDE.md	组织强制说明	是	新会话/clear/compact
User CLAUDE.md	个人全局约定	是	新会话/clear/compact
Root/project CLAUDE.md	仓库规则	是	新会话/clear/compact
Nested CLAUDE.md	子树局部知识	读入该子树时	首次加载前可生效
.claude/rules paths	按 glob 激活规则	匹配时	首次激活前可生效
MEMORY.md	跨会话索引	前 200 行或 25KB	由文件读写即时维护
Topic memory	详细经验	否，按需 Read	即时

当前 auto memory 采用透明文件，不是不可见向量库。[MEMORY.md](#) 只加载前 200 行或 25KB，topic 文件按需读取；subagent 默认不继承主会话 auto memory，除非 fork 继承父上下文，或 agent 自己启用独立 memory。^[O7]

- CLAUDE.md 写“长期稳定、每次都要遵守”的事实；不要存一次性任务状态。
- Rules 用路径约束局部规范，避免前端规则污染后端任务。
- Memory 记录已经验证的构建命令、坑点和约定；给时效性事实加日期。
- 把详细调试记录移入 topic，只在索引保留一行入口。

4. Skills 是渐进式上下文加载器

Claude Code 在会话开始只暴露 skill 的名称与描述；真正 SKILL.md 在被模型或用户调用时才注入。这是“功能可发现性”和“上下文成本”之间的折中。[disable-model-invocation](#) 或 [skillOverrides](#) 可让某些 skill 连描述也按需出现。^[O2, O16]

SDK 的 [skills](#) 选项是 context filter，而不是安全沙箱：未列出的 skill 不出现在模型清单且 Skill tool 拒绝调用，但文件仍在磁盘上，可被 Read/Bash 访问。因此 skill 文件不能存秘密。^[S5]

5. 五阶段压缩管线

阶段	机制	是否破坏历史	目的
Budget reduction	单条消息/输出预算	可能截断显示内容	阻止异常大块
History snip	裁剪旧工具输出	逻辑上替换	删除低价值噪声
Micro-compaction	细粒度缓存感知压缩	局部	延后大摘要
Context collapse	读取时虚拟投影	非破坏	保留磁盘原始事件
Auto compact	模型生成综合摘要	新边界	最后手段释放大块窗口

v2.1.88 社区分析区分 BASE、PARTIAL FROM 和 PARTIAL UP_TO：全量摘要；保留旧上下文并摘要近期；摘要到某点而保留最近原文。当前官方只承诺自动 compact 与 [/compact](#) 控制，因此三模式属于历史实现观察，不应当作为稳定外部 API。^[O2, R2]



6. 摘要的目标不是短，而是可继续

- 保留用户目标、已做决定、未完成事项、关键路径、错误与验证结果。
- 保留文件名和符号，不大段复制代码；描述改动位置与理由。
- 明确当前 permission/plan 状态，历史版本曾出现 compact 后丢失 plan mode 的 bug。
- 记录后台任务与 subagent 关系，避免摘要后重复启动。
- 禁止摘要回合调用工具，防止“总结时又改变世界”。 [O2, R2]

Compact thrashing

如果某个单文件或工具输出巨大到摘要后立即把窗口填满，自动 compact 会在少数尝试后停止并报 thrashing。修复方式是新会话、按需读取、预处理日志、用 subagent 隔离大输出，而不是继续堆摘要。 [O2]

7. Prompt caching：稳定前缀是第一性能指标

官方说明 root/user CLAUDE.md 与 output style 在会话开始读入并固定；中途修改既不立即生效，也不主动打断当前缓存前缀。nested rules/CLAUDE 在首次匹配前修改可以生效。permission mode 通常属于动态状态，应该尽量放在不会破坏稳定前缀的位置。 [O9]

易导致 cache break 的因素	控制方法
model / beta / thinking / effort	会话内减少无谓切换，必要时开新 session
system prompt / output style	固定模板，动态事实放后部
tool catalog 次序/内容	稳定排序；MCP deferred loading
时间戳与随机环境字符串	不要插入稳定前缀
插件/skills 热加载	明确刷新边界，观察 cache metrics

社区 v2.1.88 研究描述了 pre-call 记录与 post-call cache-break 诊断、相对/绝对阈值和原因分类。它提示了一种通用设计：把缓存命中当作可观测的一等指标，而非 API 账单里的事后数字。 [R2]

8. Context budget 实战

内容	建议预算策略	超标处理
CLAUDE.md	目标 <200 行，强指令在前	拆 rules/skill/topic
MCP tools	只启用任务需要的 server/tool	deferred loading / Tool Search
构建日志	先 grep/parse，再回注失败片段	hook 预处理
大文件	按符号/范围读取	subagent 总结
会话历史	一个目标一个 session	clear/compact/fork

示例 | 把稳定、可执行、可验证的规则放入上下文

```
# CLAUDE.md 建议结构
## Build and test
## Architecture invariants
## Code style
## Verification before completion
## Compact Instructions
```

9. 常见反模式

- 把整个代码库索引/README/设计文档全部塞到首轮 system prompt。
- 让每个 MCP server 的全部工具 schema 永久加载。
- 把 tool output 原样保存进摘要，导致摘要比原问题还大。
- 在 CLAUDE.md 写大量互相冲突的优先级词语，却没有可验证条件。
- 把 memory 当事实数据库，记录未经验证的猜测。
- 中途修改 output style 后以为本轮立刻生效，误判模型行为。

参考资料与证据等级

[O2] 官方 | [How Claude Code works](#)

<https://code.claude.com/docs/en/how-claude-code-works>

[O7] 官方 | [Memory / CLAUDE.md](#)

<https://code.claude.com/docs/en/memory>

[O8] 官方 | [Context window](#)

<https://code.claude.com/docs/en/context-window>

[O9] 官方 | [Prompt caching](#)

<https://code.claude.com/docs/en/prompt-caching>

[O16] 官方 | [Skills](#)

<https://code.claude.com/docs/en/skills>

[S5] 开源源码 | [SDK types and options](#)

https://github.com/anthropics/claude-agent-sdk-python/blob/v0.2.141/src/claude_agent_sdk/types.py

[R1] 社区研究 | [Dive into Claude Code v2.1.88](#)

<https://github.com/VILA-Lab/Dive-into-Claude-Code>

[R2] 社区研究 | [Claude Code Research - source analysis](#)

<https://github.com/cablate/claude-code-research>

[R3] 社区研究 | [Extracted prompt catalog](#)

<https://github.com/Piebald-AI/claude-code-system-prompts>

方法说明

官方材料用于确认当前稳定行为；SDK 源码固定在发布 tag v0.2.141；社区材料只用于解释 v2.1.88 历史源码结构。源码行号和内部函数名可能在后续版本变化，外部契约以官方最新文档为准。